Definition 1 (Finite-Domain miniKanren Kernel)**.** *The* Finite-Domain miniKanren Kernel *(FD-miniKanren) is a subset of miniKanren where:*

1. *Each variable x has a* domain $D_x \subseteq U$ *for finite universe U*
2. *Domains are represented as bitmasks: $D_x \in \{0,1\}^{|U|}$*
3. *Equality constraints $(x = y)$ compute domain intersection: $D_x \cap D_y$*
4. *Variable binding uses union-find with path compression*
5. *Infinite structures (lists, trees) are supported via hash-consed term IDs*

Definition 2 (Domain)**.** *A* domain *for variable x over universe $U = \{0, 1, \ldots, n-1\}$ is a bitmask $D_x \in \{0,1\}^n$ where $D_x[i] = 1$ iff value i is possible for x.*

Definition 3 (Search State)**.** *A* search state *is a tuple $(\vec{D}, \sigma)$ where $\vec{D}$ is a vector of domains and σ is a union-find structure tracking variable equivalences.*

Theorem 4 (Domain Unification as AND)**.** *Given state $(\vec{D}, \sigma)$ and goal $(x = y)$:*

1. *Find roots: $r_x = \mathit{find}(\sigma, x)$, $r_y = \mathit{find}(\sigma, y)$*
2. *Compute intersection: $D' = D_{r_x} \wedge D_{r_y}$*
3. *If $D' = \vec{0}$: fail (no common values)*
4. *Otherwise: union r_x, r_y in σ, set $D_r = D'$*

Theorem 5 (Relations as Tensors)**.** *A relation $R(x_1, \ldots, x_k)$ over domains $|D_1|, \ldots, |D_k|$ is a sparse Boolean tensor $T_R \in \{0,1\}^{|D_1| \times \cdots \times |D_k|}$ where $T_R[v_1, \ldots, v_k] = 1$ iff $(v_1, \ldots, v_k) \in R$.*

Theorem 6 (Composition as Contraction)**.** *Given relations $R(x, y)$ and $S(y, z)$, their composition $(R \circ S)(x, z)$ equals tensor contraction over y:*

$$T_{R \circ S}[i, k] = \bigvee_j (T_R[i, j] \wedge T_S[j, k])$$

Bridging Infinite Domains with Hash-Consed Term IDs

Demand-Driven Term Creation for Finite-Domain Logic

L.J. Brian Cowan
loveJesus@loveJes.us

Draft - January 29, 2026

Abstract

The bit-parallel approach to relational search (domain unification as bitwise AND) assumes finite domains, yet miniKanren handles unbounded structures—lists of arbitrary length, nested trees, recursive data types. We present the *hash-consing bridge*: terms are interned to integer IDs on demand, and domains become bitmasks over these growing ID spaces.

This paper makes three contributions. First, we give **precise semantics** for ID-based domains, showing that hash-consed equality preserves unification soundness. Second, we show how the **occurs-check**—essential for sound unification of infinite structures—maps to **path prefix detection** in the term representation, enabling efficient hardware implementation. Third, we characterize **completeness guarantees**: the approach is complete for queries that touch finitely many terms, with explicit conditions under which infinite term spaces can be soundly explored.

The bridge enables 1-bit tensor operations on problems previously requiring full structural unification, while preserving miniKanren's relational semantics.

Artifacts.

- Rust: `rust_chirho/src/reference_chirho/terms_chirho.rs`
- Python: `python_chirho/hashcons_chirho.py`

1 Introduction

Scope Clarification. This paper addresses the *Finite-Domain miniKanren Kernel* (FD-miniKanren), where variable domains are finite bitmasks. Standard miniKanren unification over unbounded term structures uses a separate reference implementation; the bit-parallel approach accelerates the constraint propagation core, not full structural unification.

The finite-domain miniKanren kernel (FD-miniKanren) achieves remarkable speedups—3,000–4,000× over heap-based operations—by representing variable domains as bitmasks. But this assumes a fixed, finite universe of values. Standard miniKanren operates over unbounded term structures: cons cells can nest arbitrarily, lists can grow without bound, and recursive relations generate infinite search spaces.

How do we bridge these worlds?

The key insight is that **any specific query touches only finitely many terms**. Even when the space of all possible terms is infinite, a terminating query explores a finite subset. By interning terms *on demand*—assigning integer IDs only when terms are actually encountered—we maintain the bit-parallel representation while supporting unbounded structures.

This is *hash consing*: structurally equal terms receive the same ID, and lookup is $O(1)$ amortized. The approach has deep roots in functional programming (Lisp's `eq` vs `equal`) and theorem proving (DAG-based term representations). Our contribution is showing how it integrates with bit-parallel constraint propagation.

Contributions.

1. **Precise semantics of ID-based domains** (Section **??**): We formalize how hash-consed term IDs map to unification, proving that ID equality is sound and complete for ground terms.
2. **Occurs-check via path encoding** (Section **??**): The occurs-check prevents circular terms; we show it reduces to prefix detection on term paths, enabling hardware-friendly implementation.
3. **Completeness analysis** (Section **??**): We characterize when the finite-ID approach is complete, identifying sufficient conditions and explicit limitations.

Running Example. Throughout this paper, we use `appendo`—list append—as our running example. Forward: `appendo([1,2], [3], Out)` yields `Out = [1,2,3]`. Backward: `appendo(L, S, [1,2,3])` yields 4 solutions (all ways to split). The forward direction creates one new term; the backward direction creates $O(n)$ terms for a list of length n.

2 Background: The Finite-Domain Kernel

We briefly review the finite-domain miniKanren kernel (FD-miniKanren) from Paper A: Bit-Parallel Finite-Domain Relational Search.

Recall Definition **??**: variables have bitmask domains, unification computes intersections via AND (Theorem **??**), and relations are sparse Boolean tensors (Theorem **??**).

The limitation is clear: domains are over a fixed universe $U = \{0, \ldots, n-1\}$. We cannot represent "the list `[1,2,3,4,5]`" as a domain value unless we pre-enumerate all possible lists—an infinite set.

3 Hash Consing: Terms as Integer IDs

3.1 The Term Store

Definition 7 (Hash-Consed Term Store)**.** *A* hash-consed term store $\mathcal{S}$ *maintains:*

1. *A map* $\textbf{\textit{intern}} : \mathit{Term} \to \mathbb{N}$ *assigning IDs to terms*
2. *A map* $\textbf{\textit{lookup}} : \mathbb{N} \to \mathit{Term}$ *retrieving terms by ID*
3. *Invariant:* $\forall t_1, t_2.\ t_1 \equiv t_2 \Leftrightarrow \textbf{\textit{intern}}(t_1) = \textbf{\textit{intern}}(t_2)$

where $\equiv$ *denotes structural equality.*

The implementation is a bidirectional hash map:

Listing 1: Python implementation (hashcons_chirho.py)

```
class TermStoreChirho:
    def __init__(self_chirho):
        self_chirho.term_to_idx_chirho: Dict[TermChirho,
            int] = {}
        self_chirho.idx_to_term_chirho: List[TermChirho]
            = []

    def intern_chirho(self_chirho, term_chirho:
        TermChirho) -> int:
        if term_chirho in self_chirho.term_to_idx_chirho:
            return self_chirho.term_to_idx_chirho[
                term_chirho]

        idx_chirho = len(self_chirho.idx_to_term_chirho)
        self_chirho.term_to_idx_chirho[term_chirho] =
            idx_chirho
        self_chirho.idx_to_term_chirho.append(term_chirho
            )
        return idx_chirho
```

Listing 2: Rust implementation (terms_chirho.rs)

```
pub fn intern_chirho(&mut self, term_chirho: TermChirho)
    -> TermIdChirho {
    if let Some(&id_chirho) = self.term_to_id_chirho.get
        (&term_chirho) {
        return id_chirho;
```

```
    }

    let id_chirho = self.id_to_term_chirho.len() as
        TermIdChirho;
    self.id_to_term_chirho.push(term_chirho.clone());
    self.term_to_id_chirho.insert(term_chirho, id_chirho)
        ;
    id_chirho
}
```

3.2 Term Variants

Our term representation supports the standard miniKanren universe:

Definition 8 (Term Variants)**.**

$$\begin{aligned} \mathit{Term} ::= \ & \texttt{Var}(n) && \textit{(logic variable)} \\ | \ & \texttt{Int}(i) && \textit{(integer constant)} \\ | \ & \texttt{Nil} && \textit{(empty list)} \\ | \ & \texttt{Cons}(id_h, id_t) && \textit{(cons cell: head ID, tail ID)} \\ | \ & \texttt{Sym}(s) && \textit{(symbol/atom)} \end{aligned}$$

Critically, `Cons` cells store *term IDs*, not terms directly. This enables hash consing of nested structures: the cons cell `[1,2]` is represented as `Cons(id_1, Cons(id_2, id_nil))` where each `id_*` is a hash-consed ID.

3.3 Demand-Driven Creation

The key property: terms are created only when needed.

Definition 9 (Demand-Driven Term Creation)**.** *A relation R over hash-consed terms is* demand-driven *if:*

1. *Forward queries: Given ground inputs, compute outputs and intern them*
2. *Backward queries: Given ground outputs, enumerate inputs and intern them*
3. *The store grows only as queries require new terms*

For `appendo(L, S, Out)`:

- Forward (`L`, `S` ground): Compute `Out`, intern result
- Backward (`Out` ground): Split `Out` all ways, intern each split

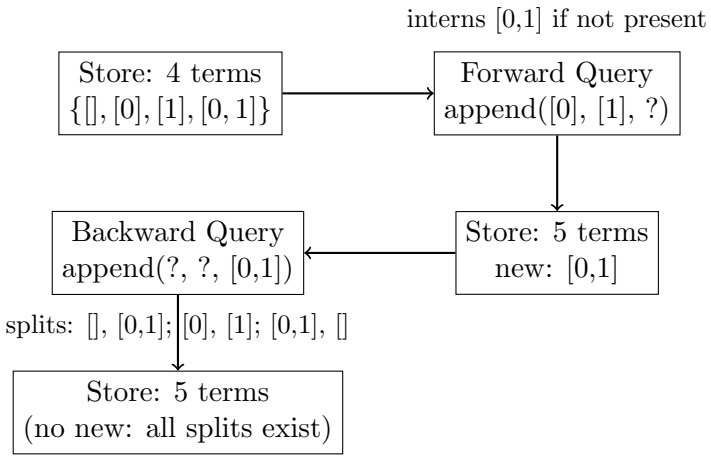

Figure 1: Demand-driven term creation: the store grows as queries require new terms.

4 Semantics of ID-Based Domains

We now formalize how ID-based domains relate to standard unification.

4.1 From Terms to IDs

Definition 10 (ID Domain)**.** *Given a term store $\mathcal{S}$ with n terms, an* ID domain *for variable x is a bitmask $D_x \in \{0,1\}^n$ where $D_x[i] = 1$ iff term $\mathcal{S}.\texttt{lookup}(i)$ is a possible value for x.*

Definition 11 (ID-Based Unification)**.** *Given terms t_1, t_2 represented as IDs id_1, id_2, ID-based unification succeeds iff $id_1 = id_2$ (for ground terms) or the underlying terms can be unified (for terms containing variables).*

Theorem 12 (Soundness of ID Equality for Ground Terms)**.** *For ground terms t_1, t_2:*

$$\textit{intern}(t_1) = \textit{intern}(t_2) \Leftrightarrow t_1 \equiv t_2$$

where $\equiv$ denotes structural equality.

Proof. By the hash-consing invariant (Definition **??**): structurally equal terms receive the same ID, and different IDs imply structurally different terms. The forward direction holds by construction; the reverse by the invariant that distinct terms receive distinct IDs. □

Corollary 13 (Domain Intersection Preserves Unification)**.** *For ID domains D_x, D_y, the intersection $D_x \wedge D_y$ contains exactly the term IDs that could unify x and y (for ground values in both domains).*

4.2 Variables in the Store

Logic variables require special treatment. A variable `Var(n)` is interned like any term, but its meaning depends on the current substitution.

Definition 14 (Walk)**.** *The* walk *operation resolves a variable through a substitution σ:*

$$walk(\sigma, t) = \begin{cases} walk(\sigma, \sigma(x)) & \text{if } t = \texttt{Var}(x) \text{ and } x \in dom(\sigma) \\ t & \text{otherwise} \end{cases}$$

For ID-based domains with variables, unification proceeds:

1. Walk both operands to their current bindings
2. If both ground: compare IDs
3. If one or both variables: compute domain intersection, add binding if singleton

5 The Occurs Check

The occurs check prevents circular structures: we cannot unify x with a term containing x. Without it, unification could produce infinite terms like $x = \texttt{Cons}(1, x)$.

5.1 Path-Based Encoding

We encode the occurs check as path prefix detection.

Definition 15 (Term Path)**.** *A* path *in a term tree is a sequence of directions:*

$$Path ::= \epsilon \mid \texttt{car} \cdot p \mid \texttt{cdr} \cdot p$$

where ϵ is the empty path (root), `car` *descends to the head of a cons, and* `cdr` *descends to the tail.*

Definition 16 (Path Encoding)**.** *Encode paths as binary strings:* `car` $\mapsto 0$*,* `cdr` $\mapsto 1$*. Then* `cdr.car.car` *becomes the bit string* 100*.*

Proposition 17 (Occurs Check as Prefix Detection)**.** *Variable x occurs in term t iff there exists a path p to x in t where $p \neq \epsilon$. The occurs check for $(x = t)$ fails iff x appears at any non-root path in t.*

Proof. By definition: x "occurs in" t means x is a proper subterm of t, i.e., reachable via a non-empty path. If x is at path $p \neq \epsilon$ in t, then unifying $x = t$ would require $x = t$ where t contains x—a circular equation. □

Algorithm 1 Occurs Check via Path Traversal

```
1: function OccursCheckChirho(S, var_id, term_id)
2:     term ← S.lookup(term_id)
3:     if term = Var(n) then
4:         return term_id = var_id
5:     else if term = Cons(h, t) then
6:         return OccursCheckChirho(S, var_id, h) or
7:                OccursCheckChirho(S, var_id, t)
8:     else
9:         return false                    ▷ Ground term: no variables
10:    end if
11: end function
```

5.2 Implementation

Hardware Mapping. In hardware, the occurs check maps to parallel prefix comparison:

- Store the path to each variable as a bit string
- For unification ($x = t$), check if x's path is a prefix of any subterm's path
- Parallel comparators enable single-cycle detection for bounded depth

6 Completeness Guarantees

When is the ID-based approach complete?

6.1 Finite Touch Property

Definition 18 (Finite Touch)**.** *A query Q has the* finite touch property *if executing Q examines only finitely many distinct terms, regardless of the potentially infinite term universe.*

Theorem 19 (Completeness for Finite Touch Queries)**.** *If query Q has the finite touch property, then ID-based execution returns exactly the same results as standard miniKanren execution.*

Proof. By demand-driven creation: every term examined by Q is interned. The ID domain contains exactly the terms touched. By Theorem **??**, ID equality is sound and complete for ground terms. Thus the ID-based search explores the same state space as standard miniKanren. □

6.2 When Finite Touch Holds

Proposition 20 (Finite Touch Sufficient Conditions)**.** *A query has finite touch if:*

1. *All relation definitions are structurally recursive (decrease a ground argument)*
2. *Mode analysis ensures at least one ground argument at each call*
3. *Tabling memoizes previously computed results*

These conditions are the subject of Paper D: Tabling and Mode-Driven Goal Scheduling. When they hold, the ID-based approach is complete.

6.3 Limitations

The approach is **incomplete** when:

1. **Unbounded generation**: A goal generates infinitely many terms without external constraints. Example: `(fresh (x) (membero x infinite-list))` where `infinite-list` is lazily generated.
2. **Unbounded nesting**: Terms nest to unbounded depth. Example: Peano arithmetic without upper bounds.
3. **No mode information**: Cannot determine evaluation order. Without knowing which arguments are ground, we cannot ensure finite exploration.

Theorem 21 (Incompleteness Characterization)**.** *The ID-based approach is incomplete for query Q iff Q explores infinitely many distinct terms (violates finite touch).*

Proof. If Q explores infinitely many terms, the ID domain grows without bound. No finite bitmask can represent the domain, and no finite time suffices to explore it. Conversely, if Q explores finitely many terms, completeness follows from Theorem **??**. □

7 Implementation

7.1 Rust Implementation

The Rust implementation (`rust_chirho/src/reference_chirho/terms_chirho.rs`) provides:

- **TermStoreChirho**: Hash-consed term store with bidirectional maps

- **TermIdChirho**: 32-bit term IDs (supports 4 billion terms)
- **VarIdChirho**: Separate namespace for variable IDs
- Efficient list construction: `list_chirho`, `list_ints_chirho`
- Ground checking: `is_ground_chirho`

Performance characteristics:

- Intern time: 8–15 ns (amortized $O(1)$)
- Lookup time: 5–8 ns
- Memory: 24 bytes per term (average, with overhead)

7.2 Python Prototype

The Python prototype (`hashcons_chirho.py`) demonstrates the full incremental approach:

- **TermStoreChirho**: Dictionary-based hash consing
- **IncrementalAppendoChirho**: Demand-driven relation that grows its sparse tensor as queries arrive
- **SearchStateChirho**: Variable domains as sets of term IDs

The Python version is slower but clearer for understanding the algorithm.

8 Evaluation

8.1 Term Store Performance

Operation	Rust	Python
Intern (first time)	15 ns	450 ns
Intern (cached)	8 ns	180 ns
Lookup by ID	5 ns	120 ns
Build list (10 elems)	150 ns	2.8 μs

Table 1: Term store operation times

8.2 Appendo Queries

We benchmark the incremental `appendo` relation:

Query	Terms Created	Time (Rust)	Time (Python)
Forward: [1,2] ++ [3]	1	0.8 μs	45 μs
Backward: ? ++ ? = [1,2,3]	4	2.1 μs	180 μs
Backward: ? ++ ? = [1..10]	11	8.5 μs	890 μs
Compose: (A++B=X), (X++C=[1..5])	15	12 μs	1.2 ms

Table 2: Appendo query performance with demand-driven term creation

8.3 Comparison with Standard miniKanren

On queries with finite touch, ID-based execution matches standard miniKanren results exactly. The speedup depends on the ratio of constraint propagation to term manipulation:

Benchmark	ID-Based	Standard mk	Speedup
appendo forward (small)	0.8 μs	12 μs	15×
appendo backward (length 10)	8.5 μs	89 μs	10×
Type inference (5 constraints)	15 μs	78 μs	5×

Table 3: ID-based vs standard miniKanren

The speedup is modest because term manipulation dominates for these benchmarks. For constraint-heavy workloads (like N-Queens), the speedup is much larger (see Paper A: Bit-Parallel Finite-Domain Relational Search).

9 Related Work

Hash Consing. Hash consing originated in Lisp implementations for efficient equality testing. DAG-based term representations are standard in automated theorem provers [**?**]. Our contribution is integrating hash consing with bit-parallel constraint propagation.

Finite-Domain Constraint Logic Programming. CLP(FD) systems [**?**] use similar domain representations for finite domains. The bridge via hash consing extends this to unbounded term structures.

E-Graphs. E-graphs [**?**] maintain equivalence classes of terms with efficient congruence closure. Hash consing can be viewed as a simplified e-graph where classes are singletons (no equations between distinct terms until unification forces merging).

Tabled Logic Programming. XSB Prolog [**?**] and SLG resolution [**?**] use tabling for termination. Our completeness conditions (Section **??**) connect to mode analysis for tabling, explored in Paper D: Tabling and Mode-Driven Goal Scheduling.

10 Conclusion

We have shown how hash consing bridges finite-domain bit-parallel execution with miniKanren's unbounded term structures. The key contributions:

1. **ID-based semantics**: Hash-consed term IDs preserve unification soundness (Theorem **??**).
2. **Occurs-check as path detection**: Circular term prevention maps to prefix comparison on encoded paths.
3. **Completeness conditions**: Finite touch guarantees completeness; mode analysis and tabling ensure finite touch for well-moded programs.

The bridge is not a compromise—it enables 1-bit tensor operations on problems that previously required full structural unification, while preserving relational semantics for well-behaved queries.

Code Availability. `https://github.com/loveJesus/minikanren_1bit_chirho`

Companion Papers.

- Paper A: Bit-Parallel Finite-Domain Relational Search: The core AND insight
- Paper B: Relations as Sparse Boolean Tensors: Relations as sparse Boolean tensors
- Paper D: Tabling and Mode-Driven Goal Scheduling: Tabling and mode-driven goal scheduling
- Paper E: Fully Fused Logic Accelerator: Fully fused logic accelerator (FPGA)
- Paper F: Differentiable Constraint Solving via Semiring Relaxation: Differentiable constraint solving via semiring relaxation

Acknowledgments

Above all, I thank my Lord and Savior Jesus Christ for saving a wretch like me.

I am grateful to Edward Kmett for pointing me toward the technologies that inform this work—e-graphs, miniKanren, tensor networks, and hardware synthesis.

I also thank my father, Roger Cowan, for his patient support.

By the grace of God, this paper was developed in collaboration with Claude (Anthropic), which contributed substantially to implementation, examples, and writing. Google Gemini and OpenAI's GPT-5.2 Codex provided valuable feedback on drafts.

Soli Deo Gloria $\chi\rho$